Infrastructure & Workflow

> Note: Below are sample Development Process & Workflow. No templates are required for this; feel free to use and include applicable information for your internal Dev Process and Workflow.
> You can refer to the example below, researching for other dev process and workflow is also encouraged.

SAMPLES FOR YOUR REFERENCE

Environments Overview



Environments Overview

Two of four exist. The other two are specified so that building them later is implementation, not decision-making.

Concern	Local EXISTS	Test EXISTS	Staging NOT BUILT	Production NOT BUILT
App hosting	your own machine	<<< PaaS >>> free tier	same as Test	same as Test
Database	local instance	<<< managed >>> free tier	same engine + version	same engine + version
Email	logged to console	<<< provider >>> sandbox	provider, real sends	provider, real sends
Catalogue API key	shared dev key	OWN key	own key	own key
Deployed from	working copy	develop, on merge	main, on tag	tag + approval
Who may deploy	you	any dev, via merge	DevOps	DevOps + PM
Data	seed/dev.sql	seeded, resettable	anonymised copy	real
Monthly cost	£0	£0 · sleeps when idle	~£7	~£7

Test has its OWN catalogue API key. The provider's 40-requests-per-10-seconds limit is per key, so a CI burst on a shared key would eat the budget the Saturday demo needs. One line of configuration; one ruined demo avoided.

Staging and Production are held as tickets MT-201 and MT-202. Every row above is filled in for them, so nobody has to decide any of it at 11pm.

App hosting

Provider	`<<< PaaS >>>` free tier
Runtime	The same build artifact in every environment
Test URL	`<<< https://movietrack-test.example.app >>>`
Sleeps when idle	after 15 mins
Scaling	Only One instance.

Database hosting

Provider	`<<< managed database >>>` free tier
Engine	`<<< engine >>>`, **use the same major version in CI, local, and test**
Size	1 GB storage — enough for ~2,000 films per account
Connections	Free tier caps connections — use a connection pool; do not open one per request
Backups	Provider daily snapshot with 7-day retention. **See "Backup and Restore" below**

Other services

Service	Used for	Notes
Email	Address verification, password reset	`<<< provider >>>` free tier, daily send cap. Console-logged in Local
Film catalogue API	Search and metadata	40 requests / 10 seconds **per key**
CI	Pipeline and secret store	`<<< CI provider >>>` free tier
DNS + TLS	Test URL	Provider-managed certificate with auto-renew. Nothing to do until a custom domain exists

Test has its own catalogue API key. The provider's limit is per key, so a CI burst on a shared key could use the budget needed for the Saturday demo. One line of configuration prevents a ruined demo.

Deployment Process

Sprint release

Every merge into `develop` deploys to Test automatically, several times a day. No approval, no tag, no announcement — that is the point.

```
merge to develop → pipeline stages 1-6 → migrate → deploy → smoke check
```

The smoke check calls `/health` and asserts it returns 200 **and the commit sha it just deployed**. A deploy that does not verify itself is one you hear about from a student on Saturday morning.

Final release

At the end of a sprint, `develop` goes to `main` and is tagged.

1. **Open the release pull request**, `develop` → `main`. The description lists every ticket included — that list becomes the release notes.
2. **Check the pre-deploy list** (below).
3. **Merge and tag** `v0.<n>.0`.
4. **Approve.** DevOps and the PM, named in the pull request. This is the only manual gate in the process, and it exists because it is the only deploy anyone outside the team sees.
5. **Deploy** — the tag goes to Staging, then to Production on approval. *(Both pending MT-201 and MT-202; today a final release stops at Test.)*
6. **Verify** — smoke check, then the three main journeys by hand.
7. **Announce** in the Teams channel before the Sprint Review.

Pre-deploy checklist

- [ ] Pipeline green on `develop`, including the authorization stage
- [ ] Migrations reviewed by someone other than the author and backward compatible
- [ ] No new configuration value missing from the target environment
- [ ] No secret added to the repository, including in a reverted commit
- [ ] Demo dry run done — the "it worked on my machine" catch
- [ ] Someone available for the next hour

> **One artifact, built once, promoted unchanged.** A final release deploys the same bytes that were on Test all sprint. If Test and Production needed different builds, an environment difference has leaked into the code.

Configuration Management

Variable	Local	Test	Source	Notes
`DATABASE_URL`	local instance	managed instance	secret store	—
`CATALOGUE_BASE_URL`	provider sandbox	provider sandbox	env var	—
`CATALOGUE_API_KEY`	shared dev key	**Test-only key**	secret store	Per-key rate limit
`CATALOGUE_RATE_LIMIT`	`35/10s`	`35/10s`	env var	Below the published 40, for headroom
`CACHE_TTL_HOURS`	`24`	`24`	env var	ADR-003
`SESSION_IDLE_MINUTES`	`30`	`30`	env var	—
`BCRYPT_COST`	`12`	`12`	env var	Minimum also enforced in code
`EMAIL_API_KEY`	unused	sandbox key	secret store	—
`LOG_LEVEL`	`debug`	`info`	env var	—

Rules:

1. Read once at start-up and **validate then**. A missing required value stops the process loudly; it never defaults silently.
2. `.env.example` is committed with **every key and every value empty**, so a new joiner knows what to fill in without ever seeing a real one.
3. Changing a value in an environment is a change like any other — say so in the Teams channel, because a config change that nobody announced looks exactly like a bug.

Secrets

Stored in	The CI provider's secret store. Nowhere else
Who can read	DevOps. Others get values when they need them, individually
Rotated	End of the module, or **immediately on exposure**
Masked in logs	Explicitly. Never `echo` a variable to debug it

Database Migrations

- Every schema change is a **reviewed file in the repository**. No hand-edited databases, ever, including on Test.
- Migrations run against an **empty database on every CI run**, which makes every build a free test of the migration path.
- They are applied by the pipeline **before** the new code starts, never by a person.
- **Backwards-compatible for one release** (see Rollback).
- A destructive change is split in two: release *n* stops using the column, release *n+1* drops it.

Backup and Restore

What	Provider daily snapshot of the database

Retention	7 days
Covers	All user data. Not the film cache, which is disposable by design
Restore tested	**Once, before the Sprint 2 review. Write down how long it took**

An untested restore is a belief, not a backup. The test is cheap — restore into a scratch database and confirm a seeded account is intact — and it is the only way to find out that the snapshot excludes something before you need it.

Note also that a 7-day snapshot window and the 7-day erasure obligation interact: an account deleted today still exists in snapshots for a week, which is why the erasure requirement is written as *within* seven days rather than immediately.

Monitoring and Health

Signal	Today	Sprint 3
Liveness	`GET /health` → 200 + commit sha + database check	unchanged
Catalogue state	Reported by `/health`, **not** counted as unhealthy — a catalogue outage is degraded, not down	Alert on sustained failure
Logs	Structured, one line per request, with a request id. No personal data, no secrets	Searchable retention
Uptime	**Not measured**	External check every 5 minutes
Alerting	**None — someone has to notice**	Error-rate alert

"Someone has to notice" is an honest answer for a ten-week project and a bad one for anything that outlives it. It is written down so that nobody discovers it during an incident.

Access, Cost and Limits

Role	Local	Test	Staging	Production
Dev	own	deploy + logs	—	—
QA	own	deploy + logs + data reset	read logs	—
DevOps	own	full	full	full
PM / BA / SA	own	read	read	read

Cost today: £0. Everything is on free tiers. Projected once Staging and Production exist: **about £14 a month**, and nobody has yet agreed to pay it — which is a decision needed before Sprint 3, not a discovery to make during it.

Free-tier limits worth knowing before they bite:

Limit	Value	What happens at the limit
App instance	sleeps after 15 min idle	Slow first request; excluded from performance targets
Database	1 GB, low connection cap	Pool connections; do not open one per request
Email	daily send cap	Verification and reset silently stop working — check the cap against expected registrations
Catalogue	40 req / 10 s per key	HTTP 429; the cache and limiter exist to stay under it

CI	monthly build minutes	Builds queue. Keep the pipeline under ten minutes